

Introdução

Definição

É um programa (que está sempre a correr no núcleo) que actua como intermediário entre o utilizador e o hardware.

- Gere o hardware, ou seja, coordena a utilização do hardware entre os vários programas em execução;
- Disponibiliza o ambiente para as aplicações serem executadas de maneira a que o utilizador tenha um ambiente para poder interagir com os programas;
 1. Fornece um ambiente de interação com o utilizador
 2. Fornece um ambiente para execução de programas
 3. Detecta e trata erros dos programas e/ou utilizadores
 4. Protecção entre os vários utilizadores do sistema
- Detecta e trata erros dos programas/utilizadores;
- Protecção entre os vários utilizadores do sistema operativo

Arquitectura de um SO

- Sistemas de processador único – um único CPU executa todas as tarefas.;
- Sistemas de multiprocessador– vários CPUs partilham tarefas e memória (melhor desempenho e fiabilidade).

Estrutura do SO

Multiprogramação

- Um único utilizador não consegue ocupar o CPU e os dispositivos de E/S sempre ocupados;
- Organiza os jobs (tarefas) de maneira a que o CPU tenha sempre um para executar;
- Mantém alguns jobs em memória;
- Quando um job que está a ser executado precisa de uma ação de E/S, é colocado em espera e entra outro job em execução;

TimeSharing (multi-tarefa)

- Multiprogramação e interação com o utilizador;
- A CPU alterna rapidamente entre tarefas;
- Resposta < 1 segundo, dando a ilusão de execução simultânea.

Funcionalidades do SO

Processos

- É um programa em execução;
- Precisa de recursos tais como dados, memória, CPU para realizar a sua tarefa;
- Quando termina liberta os recursos.
- É sequencial: as instruções são executadas uma após outra (sem paralelismo interno).

Sistema tem vários processos

- Processos de utilizador;
- Processos de SO.

Funções

- Cria, termina, suspende os vários processos;
- Sincroniza, comunica e faz o escalonamento.

Programa
conjunto de linhas
de código

Processo
programa em
execução



1. Gestão de Processos

- Processo = programa em execução (com contador de programa, registos, memória, etc.);
- Funções:
 - Criar, terminar, suspender e retomar processos;
 - Sincronização e comunicação entre processos;
 - Escalonamento (decidir qual processo usa a CPU).

Tipos:

- Independentes – não afetam outros;
- Cooperativos – partilham dados (usando memória partilhada ou troca de mensagens).

2. Gestão de Memória

- Decide quem usa a memória e quando;
- Move processos entre memória e disco;
- Protege o espaço de cada processo.

3. Gestão de Ficheiros e Armazenamento

- Controla criação, leitura, escrita, eliminação de ficheiros e diretórios;
- Gere permissões e acessos;
- Organiza os dados em sistemas de ficheiros.

4. Gestão de E/S (Entrada/Saída)

- Coordena controladores e drivers;
- Implementa operações como read, write;
- Usa buffers e interrupções para comunicação eficiente.

5. Comunicação entre Processos

- Memória Partilhada – processos acedem à mesma área de memória (requer sincronização);
- Troca de Mensagens – usam send() e receive():
 - Pode ser direta (entre processos) ou indireta (por mailbox);
 - Pode ser síncrona (bloqueante) ou assíncrona;
 - Pode ter buffer limitado ou ilimitado.

6. Proteção e Segurança

- Controla acessos, autenticação e isolamento de processos;
- Evita corrupção de dados e acessos indevidos.

7. Detecção de Erros

- O SO monitoriza e trata falhas de hardware ou software;
- Mantém registos de utilização (tempo de CPU, memória, ficheiros abertos).

Ambientes computacionais

- Modelos tradicionais;
- Modelos móveis
- Modelos de Tempo-Real
- Modelo cliente-servidor;
- Modelo peer-to-peer;
- Computação na nuvem.

Tipos de Sistemas Operativos

- Sistemas tempo-real: Sistemas de controlo industrial;
- Sistema interactivo: Tempo de resposta ao utilizador;
- Sistema transaccional: Acesso a recursos partilhados;
- Sistema batch : Jobs, submissão de jobs;
- Sistema embebidos: Cada vez mais comuns e com menos interface;
- PDAs, smart phones: recursos limitados.

Estruturas dos Sistemas Operativos

Interfaces com o utilizador

Interface Gráfica (GUI – Graphical User Interface)

- Interface amigável baseada em ícones, janelas, menus e apontadores (rato, toque, teclado).
- Exemplo: Windows, macOS, GNOME, KDE.
- Em dispositivos móveis, a GUI evolui para incluir toques, gestos, comandos de voz e teclado virtual.

Linha de Comandos (CLI – Command-Line Interface)

- Permite executar comandos diretamente (como mkdir, ls, nano, python script.py)

Serviços relacionados com o utilizador

O sistema operativo oferece funcionalidades que permitem executar e controlar programas, gerir ficheiros, e comunicar entre processos.

1. Execução de programas

- Carregar o programa em memória, executar e terminar;
- Reportar erros de execução.

2. Operações de Entrada/Saída (E/S)

- Controlar dispositivos e periféricos;
- Interagir com ficheiros (ler, escrever, copiar, eliminar, etc.).

3. Manipulação do sistema de ficheiros

- Criação, leitura, escrita, remoção e pesquisa em diretórios;
- Controlo de acesso e permissões.

4. Comunicações

- Troca de dados entre processos (no mesmo computador ou em rede);
- Feita por memória partilhada ou troca de mensagens.

5. Detecção de erros

- O SO deve detetar e corrigir erros (de hardware, programas ou utilizadores);
- Mantém registos e logs de eventos anormais.

Outros serviços:

Alocação de recursos

- Quando múltiplos utilizadores/processos usam o sistema em simultâneo, o SO distribui:
 - CPU, memória e armazenamento (com algoritmos específicos);
 - Periféricos, controlando pedidos e filas de espera.

Accounting / Logging

- Guarda o histórico de ações dos utilizadores e a utilização dos recursos.

Proteção e Segurança

- Garante que informação sensível não é acedida ou modificada indevidamente;
- Requer:
 - Autenticação prévia de utilizadores;
 - Cifra de dados armazenados ou transmitidos;
 - Controlo de acesso a ficheiros, dispositivos e recursos;
- A segurança só é eficaz se houver coerência entre todas as medidas de proteção.

Interface com o sistema operativo– System Calls

As system calls são o meio de comunicação entre os programas de utilizador e o núcleo do SO.

O que são?

- São funções especiais que pedem serviços ao kernel;
- As aplicações não interagem diretamente com o hardware — pedem ao SO para fazê-lo através destas chamadas.

Como o SO processa pedidos de diferentes linguagens?

- O SO fornece uma Interface de Programação (API), geralmente em C, C++ ou Java;
- As aplicações chamam funções de alto nível que, por sua vez, chamam as system calls adequadas.

Vantagens de usar APIs em vez de system calls diretas

- Portabilidade – o mesmo código pode correr em diferentes plataformas;
- Abstração – o programador não precisa conhecer detalhes do kernel;
- Otimização – as APIs incluem funcionalidades adicionais que melhoram o desempenho.

Implementação e funcionamento interno

- Cada system call tem associado um número único.
- O sistema operativo mantém uma tabela indexada por esse número.
- Quando uma aplicação invoca uma API (como `read()` ou `open()`), a biblioteca:
 1. Traduz o pedido para o número da system call;
 2. Passa os parâmetros para o kernel;
 3. O kernel executa a funcionalidade correspondente;
 4. Devolve o estado e o valor de retorno (sucesso, erro, número de bytes, etc.).

O programa que faz a chamada não precisa de conhecer a implementação interna — só precisa de saber como usar a API. A biblioteca esconde os detalhes técnicos do kernel.

Passagem de parâmetros às system calls

O kernel precisa dos parâmetros da função. Existem várias formas de os passar:

1. Registos da CPU:
 - Rápido, mas limitado (poucos registos disponíveis).
2. Tabela em memória:
 - Parâmetros colocados numa tabela;
 - Passa-se apenas o endereço da tabela num registo;
 - Usado por sistemas como Linux e Solaris.
3. Stack (pilha):
 - A biblioteca faz push dos parâmetros na stack;
 - O kernel faz pop para os ler.

Desenho e implementação

Não existe uma receita única. Várias alternativas com provas dadas. Estrutura interna varia muito

Para o utilizador:

- Facilidade de uso e conveniência;
- Fiabilidade e segurança;
- Bom desempenho.

Para o sistema:

- Facilidade de implementação e manutenção;
- Flexibilidade (adaptação a novos hardwares);
- Eficiência e robustez.

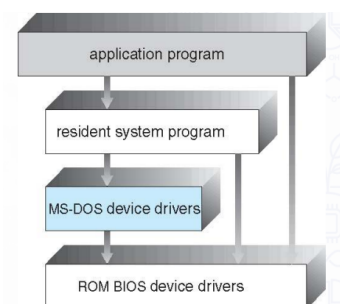
Princípio da separação entre política e mecanismo:

- Política: decisão sobre o que fazer; fim a atingir (ex.: qual processo tem prioridade).
- Mecanismo: como o fazer; implementar a política (ex.: algoritmo que muda de processo).

Estruturas Internas dos SOs

a) Estrutura simples – MS-DOS

- Foco: máximo de funcionalidades no mínimo de espaço.
Não é modular → funções misturam-se.
- Sem separação entre interfaces e funcionalidades.
- Consequência: difícil manutenção e pouca proteção entre processos.



b) Estrutura em Camadas

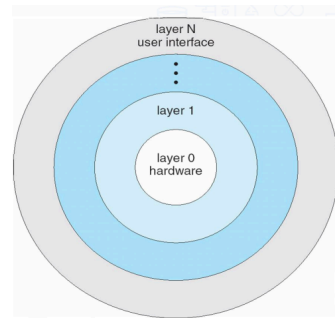
- O SO é dividido em várias camadas ou níveis hierárquicos.
- Cada camada é construída sobre a anterior:
 - Camada mais baixa - hardware;
 - Camada mais alta - interface com o utilizador.
- Cada camada usa apenas serviços das camadas inferiores.

Vantagens:

- Modularidade → facilita desenvolvimento e debugging;
- Clareza → fácil identificar onde ocorrem erros

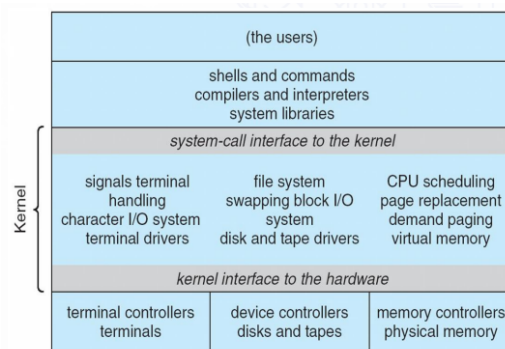
Desvantagem:

- Perda de eficiência, devido às chamadas sucessivas entre camadas.



c) Estrutura Monolítica – UNIX original

- Baseado em duas partes:
 - Programas de sistema
 - Kernel – contém toda a funcionalidade abaixo das system calls.
- O kernel gere:
 - Processos, memória, ficheiros, redes, e funções de baixo nível.
- Problema: o sistema é pouco modular, difícil de expandir e manter.



d) Estrutura de Microkernel

- Surge como solução ao excesso de código no kernel.
- Apenas o essencial fica no núcleo:
 - Gestão de processos e memória;
 - Comunicação entre processos.

Vantagens:

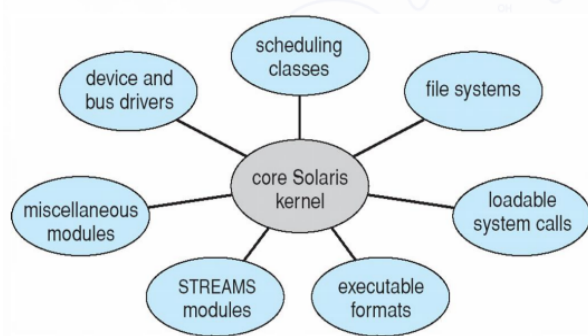
- Fácil de estender (novos serviços no modo utilizador);
- Mais seguro e fiável, pois a maioria dos serviços corre em modo utilizador.

Desvantagem:

- Desempenho inferior, pois a comunicação entre processos passa pelo kernel.

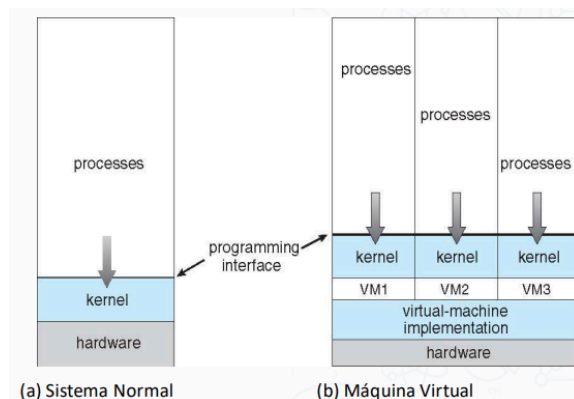
e) Estrutura Modular

- Usada nos SOs modernos (Linux,, Windows NT, etc.);
- Cada componente (módulo) é isolado e comunica por interfaces bem definidas;
- Módulos podem ser carregados dinamicamente no kernel;
- É uma combinação das ideias de camadas e microkernel:
 - Mantém eficiência, mas continua flexível e expansível.



f) Máquina Virtual (VM)

- Estende o conceito de camadas: encapsula o hardware e o SO.
- Cada VM fornece uma interface idêntica ao hardware físico.
- Várias máquinas virtuais podem partilhar os mesmos recursos físicos.



Vantagens:

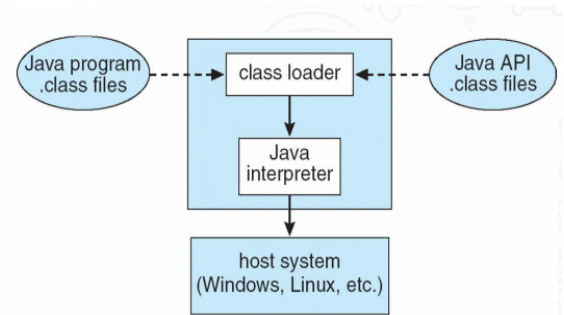
- Isolamento Segurança;
- Testes e desenvolvimento em ambientes independentes;
- Aproveitamento eficiente dos recursos físicos.

g) Máquina Virtual Java (JVM)

- Permite que programas Java corram em qualquer plataforma;
As aplicações usam a Java API, ignorando o hardware;
- O compilador Java gera bytecode;
- Um verificador de bytecode assegura que o código é seguro (proteção de memória);
O bytecode é executado:

- Por um interpretador, ou
- Por um JIT compiler (Just-In-Time), que compila em tempo real.

A JVM é, portanto, uma máquina virtual portátil, ideal para hardware limitado (como telemóveis).



Processos

Espaço de memória de um processo

Cada processo tem o seu próprio espaço de endereçamento (isolado dos outros).

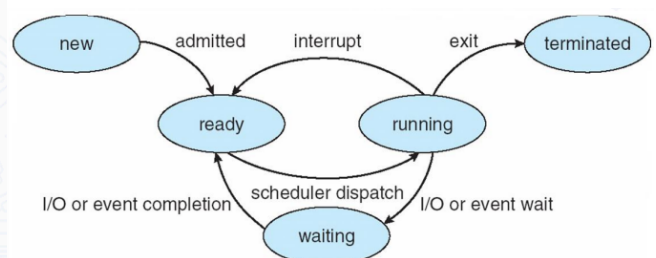
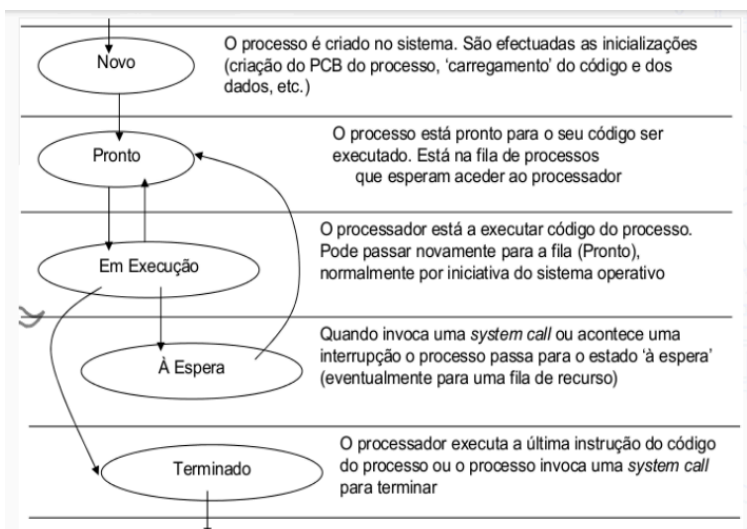
Secção	Conteúdo	Descrição
Text (código)	Instruções do programa	Parte executável
Data	Variáveis globais	Pode ter dados inicializados e não inicializados
Heap	Dados dinâmicos	Memória alocada em tempo de execução
Stack	Dados temporários	Parâmetros de funções, endereços de retorno, variáveis locais

- A stack e a heap crescem em sentidos opostos, mas o SO impede sobreposição entre elas.

Estados de um Processo

Durante a sua vida, um processo passa por vários estados:

Estado	Descrição
New (novo)	Está a ser criado
Ready (pronto)	Aguardando CPU
Running (em execução)	A ser executado no processador
Waiting (bloqueado)	À espera de evento (ex: E/S)
Terminated (terminado)	Finalizou a execução



Árvore de Processos

Os processos formam uma árvore hierárquica:

- O pai cria filhos, e estes podem criar netos, etc.

PCB – Process Control Block

O PCB é a estrutura de dados que guarda toda a informação de um processo:

Informação armazenada	Exemplos
Estado do processo	running, ready, waiting...
Identificador (PID)	número do processo
Contador de programa (PC)	próxima instrução a executar
Registos do CPU	contexto atual
Dados de escalonamento	prioridade, tempo de CPU usado
Espaço de memória	limites, tabelas de páginas
Operações de I/O	ficheiros e dispositivos abertos
Accounting	tempo de execução, dono, UID

Escalonamento de Processos

O que é o escalonamento?

- Os processos competem pelos recursos, especialmente pela CPU.
- O escalonador (scheduler) decide:
 - Qual processo será executado a seguir;
 - Quando será interrompido;
 - Em que CPU (em sistemas multiprocessador)

Filas de Escalonamento

O SO organiza os processos em várias filas:

Fila	Descrição
Ready Queue	Processos prontos para executar (à espera da CPU)
Wait Queues	Processos à espera de eventos (como E/S ou filhos terminarem)
Fila por dispositivo	Cada periférico tem a sua própria fila de espera

Comutação de Processos e Mudança de Contexto

Quando o scheduler (escalonador) retira um processo do estado running e o substitui por outro.

Mudança de Contexto (PCB) :

- Quando há comutação entre dois processos, o SO deve salvar o estado do antigo processo na PCB deste e carregar o estado do novo processo da sua PCB
- Esta operação é fundamental para que os processos possam voltar para o estado running sem perder o contexto de execução
- O tempo de mudança de contexto é um overhead. O sistema não está a fazer nada de útil para o utilizador durante a comutação

Passos:

1. O SO guarda o estado do processo atual no seu PCB (program counter, registos, etc.);
2. Carrega o estado do novo processo a ser executado;
3. A CPU continua a execução como se nada tivesse acontecido.

Operações sobre Processos

O SO fornece operações de criação, suspensão e terminação de processos.

Criação de Processos

Um processo pai pode criar processos filhos. Os filhos podem, por sua vez, criar outros processos, formando uma árvore de processos. O processo pai e o processo filho tornam-se idênticos, com exceção de:

- PID (identificador do processo);
- Endereço da memória (cada um tem a sua própria cópia).

Recursos dos filhos:

São atribuídos pelo SO ou herdados do pai (memória, CPU, ficheiros, etc.).

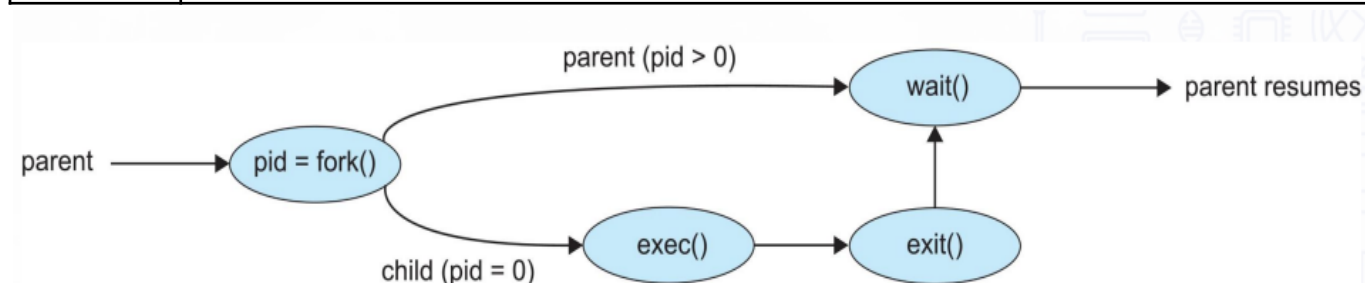
Após a criação: O pai pode:

- Continuar em paralelo com o filho;
- Ou esperar que o filho termine (`wait()`).

Partilha de recursos: Pai e filho podem:

- Partilhar todos os recursos;
- Partilhar apenas alguns;
- Ou não partilhar nada.

Chamada	Função
<code>fork()</code>	Cria um novo processo como cópia do pai; ambos continuam a correr o mesmo código.
<code>exec()</code>	Substitui o código e o espaço de memória do processo pelo de um novo programa.
<code>wait()</code>	O pai suspende-se até que o filho termine.
<code>exit()</code>	Termina o processo e liberta todos os recursos.
<code>abort()</code>	O pai força a terminação imediata de um filho.



-Um processo é órfão quando o pai termina antes dele (continua executando normalmente).

-Um processo torna-se zombie quando ele terminou sua execução, mas o pai ainda não chamou `wait()` para ler o seu status.

Comunicação entre Processos

Tipos de processos concorrentes

Processos Independentes:

- Não afetam nem são afetados por outros processos.
- Não partilham dados nem recursos com outros.
- Exemplo: os processos filhos criados nos exemplos anteriores com `fork()`, que não trocam informação com o pai.

Processos Cooperativos:

- Podem afetar ou ser afetados por outros processos.
- Partilham pelo menos um recurso:
 - zona de memória, ficheiros, resultados, etc.

Motivação para processos cooperativos

1. Partilha de informação
 - Múltiplos processos podem precisar aceder à mesma informação (ex.: ficheiro partilhado).
2. Melhoria de desempenho
 - Divisão de tarefas em paralelo → melhor utilização de CPU e dispositivos.
3. Modularidade
 - O sistema pode ser dividido em módulos independentes (cada um num processo).
4. Conveniência
 - Permite que o utilizador trabalhe em várias tarefas simultaneamente.

Modelos de Comunicação entre Processos

Há dois grandes modelos de IPC:

Modelo	Descrição	Exemplo típico
Memória partilhada	Os processos comunicam escrevendo/lendo numa zona de memória comum.	Variáveis partilhadas entre processos
Troca de mensagens	Os processos trocam mensagens (send/receive), sem partilhar memória.	Comunicação cliente-servidor, redes

Comunicação baseada em Memória Partilhada

- Os processos pedem ao SO para criar uma zona de memória partilhada.
- Essa zona é mapeada no espaço de endereçamento de ambos os processos.
- Após a criação, os próprios processos gerem o conteúdo (não o SO).

Funcionamento:

1. Processo A pede ao SO para criar a memória partilhada.
2. Processo B “liga” essa zona ao seu espaço de endereçamento.
3. Ambos comunicam lendo/escrevendo diretamente nessa memória.
4. Quando terminam, podem “desligar-se” da zona.

Gestão e Sincronização

Uma vez criada, a zona deixa de ser gerida pelo SO — os processos controlam o que armazenam.

Requer sincronização manual, para evitar conflitos:

- write–write: dois processos tentam escrever ao mesmo tempo;
- read–write: um tenta ler enquanto o outro ainda escreve.

Vantagens	Desvantagens
Muito rápida (acesso direto à RAM)	Requer sincronização manual
Ideal para grandes volumes de dados	Gasta tempo na criação do espaço
Pouca intervenção do kernel (só na criação)	Usa espaço do endereço dos processos
Operações simples: criar, ligar, ler, escrever	

Comunicação baseada em Troca de Mensagens

-A comunicação é feita trocando mensagens entre processos.

-As mensagens são copiadas (não há memória comum).

-Operações básicas:

- send(message) → envia uma mensagem;
- receive(message) → recebe uma mensagem.

-Os processos precisam acordar o formato e a sequência das mensagens.

-Mensagens fora do formato esperado são descartadas.

Vantagens	Desvantagens
Simple e segura para pequenas mensagens	Mais lenta (envolve o kernel)
Ideal para sistemas distribuídos (rede)	Requer protocolos complexos
Evita partilha direta de memória	

Questões de implementação

Quando dois processos (A e B) comunicam, é necessário definir:

- Como os canais (links) são criados;
- Quantos links podem existir entre processos;
- Se o link é unidirecional ou bidirecional;
- Se o tamanho das mensagens é fixo ou variável;
- Qual é a capacidade (quantas mensagens podem estar “em espera”).

Implementação de canais de comunicação

Aspeto físico	Aspeto lógico
Barramento de hardware	Comunicação direta ou indireta
Memória partilhada	Comunicação síncrona ou assíncrona
Rede (TCP/IP, etc.)	Buffering automático ou explícito

Comunicação

Direta:

- Pode ser simétrica (um canal exclusivo por par de processos);
- Ou assimétrica (vários emissores → um recetor).

Indireta:

- Mensagens vão para uma caixa de correio (mailbox);
- Os recetores retiram as mensagens dessa caixa;
- É necessário lidar **com conflitos de receção simultânea**.

Sincronização

Tipo	Descrição	Efeito
Bloqueante (Síncrono)	O emissor ou recetor esperam até o evento ocorrer	Comunicação garantida
Não-bloqueante (Assíncrono)	O emissor/recetor não esperam (podem continuar execução)	Risco de perda de mensagens

Buffering

As mensagens podem ficar temporariamente em filas (buffers).

Tipo	Capacidade	Comportamento
Zero (0)	Nenhuma mensagem armazenada	Emissor bloqueia até recetor ler
Limitada (n)	Até n mensagens	” bloqueia quando fila cheia
Ilimitada	Virtualmente infinita	Emissor nunca bloqueia

IPC no Linux

Mecanismos principais

1. Pipes sem nome
2. Filas de mensagens (Queue)
3. Memória partilhada
4. Mensagens pela rede (Sockets)

Threads

O que é uma thread?

- Uma thread é um fluxo de execução independente dentro de um processo.
- Cada processo tem pelo menos uma thread principal.
- Num processo multithreaded, existem várias threads que:
 - Partilham o mesmo espaço de endereçamento (código, dados, ficheiros);
 - Têm registos e pilha próprios.

O escalonador (scheduler) do SO passa a trabalhar ao nível das threads e não apenas dos processos.

Vantagens do uso de threads

- Melhor desempenho (criação mais rápida que processos).
- Aproveitamento de CPUs multicore.
- Partilha fácil de memória e recursos.
- Resposta mais rápida em aplicações interativas.

Dificuldades no uso de threads

- Divisão das tarefas nem sempre é simples.
- Difícil manter equilíbrio de carga.
- Requer sincronização entre threads que acedem aos mesmos dados.
- Debug e testes tornam-se mais complexos.

Processo vs Thread

Processo

Espaço de memória independente
Comunicação lenta (via IPC)
Elevado custo de criação e mudança de contexto
PCB (Process Control Block) próprio

Thread

Partilha o mesmo espaço de memória
Comunicação direta via variáveis comuns
Leve e rápida de criar e alternar
Contexto menor: PC, registos e stack

Paralelismo e Concorrência

- Concorrência: várias threads partilham o CPU, executando “em simultâneo” (troca rápida).
- Paralelismo: várias threads executam em diferentes núcleos de CPU.
- Multicore / Multithreaded CPUs: um processador físico pode simular vários processadores lógicos (ex.: 2 núcleos com 2 threads = 4 processadores lógicos)

Modelos Multithreading

Relação entre threads do utilizador e threads do núcleo

Modelo	Descrição	Vantagens / Limitações	Desvantagens
Many-to-One	Várias threads de utilizador mapeadas numa só thread de núcleo.	-Podem ser executados em SO que não suportem threads -Mais rápido do que kernel threads -Cada processo pode ter o seu próprio algoritmo de escalonamento de threads	-Gestão de falta de paginas, o processo bloqueia todo mesmo que as outras threads possam continuar -A libertação do CPU tem de ser feito pela própria thread que o está a utilizar -Não usufrui das vantagens de multiprocessador
One-to-One	Cada thread de utilizador tem uma thread de núcleo.	Suporta paralelismo, mas cria mais overhead.	-Criação de um thread nível núcleo demora mais tempo (Desempenho) -Não há reciclagem de threads (Desempenho)
Many-to-Many	Associam threads de nível de utilizador a um nr menor ou igual de threads de nível de núcleo	Controla nº de threads no kernel, melhor desempenho.	
Two-Level	Permitem associar threads de nível de utilizador a um nr menor ou igual de threads de nível de núcleo mas também uma thread de utilizador a uma thread de núcleo	Usado em UNIX (Solaris, HP-UX, etc.).	

Thread Pools

- Um thread pool é um conjunto fixo de threads criadas antecipadamente.
- As threads ficam à espera de tarefas numa fila.
- Quando uma tarefa chega, é atribuída a uma thread livre.

Vantagens:

- Evita criar/destruir threads repetidamente.
- Controla o número máximo de threads → previne sobrecarga.
- Melhora o desempenho em servidores.

Suporte a Multithreading nos SOs modernos

Tipo	Descrição	Exemplos
Threads de utilizador	Geridas em espaço de utilizador	Bibliotecas POSIX Pthreads.
Threads de núcleo	Geridas diretamente pelo kernel.	Linux, Windows, macOS

Escalonamento de Processos

Multiprogramação

- O objetivo do escalonamento é manter o CPU sempre ocupado, executando um processo útil.
- Quando um processo fica em espera (por exemplo, devido a uma operação de I/O), o SO escolhe outro processo pronto (ready) para usar o CPU.
- Isto aumenta:
 - A utilização do CPU;
 - O throughput (número de processos concluídos por unidade de tempo);
 - A eficiência geral do sistema.

Ideia essencial: Nunca deixar o processador ocioso — se um processo bloqueia, o SO dá CPU a outro.

Ciclo de execução dos processos

Cada processo alterna entre:

- Rajadas de CPU (CPU bursts) — períodos de processamento;
- Rajadas de I/O (I/O bursts) — períodos de espera por operações de entrada/saída.

Tipos de processos (conforme os ciclos)

Tipo	Características	Exemplo
Interativo	Muitas rajadas curtas de CPU e muita I/O	Aplicações de utilizador
Batch	Pouca I/O e rajadas longas de CPU	Processamento em lote

A caracterização das rajadas ajuda o SO a escolher o algoritmo de escalonamento ideal. O escalonador (scheduler) é chamado quando:

1. Um processo passa de **running** → **waiting** (ex.: pede I/O);
2. Um processo passa de **running** → **ready** (interrupção ou preempção);
3. Um processo passa de **waiting** → **ready** (termina I/O);
4. Um processo **termina**.

Tipos de Escalonamento

Tipo	Características	Situações
Não preemptivo	O processo liberta o CPU voluntariamente (termina ou entra em espera).	Situações 1 e 4
Preemptivo	O SO retira o CPU à força (interrupção, prioridade, quantum expirado).	Situações 2 e 3

A maioria dos sistemas modernos é preemptiva, pois aumenta a responsividade.
Problemas no escalonamento preemptivo

- Quando dois processos partilham dados, uma mudança abrupta de contexto pode causar inconsistência:
 - O processo A escreve parcialmente;
 - O SO interrompe A e dá CPU a B;
 - B lê dados inconsistentes.

Solução: sincronização.

Expedidor (Dispatcher)

- É o módulo que atribui o CPU ao processo selecionado pelo escalonador.
- Tarefas do dispatcher:
 - Guardar o estado do processo atual (context switch);
 - Trocar para modo utilizador;
 - Recomeçar o processo seguinte.
- Latência de despacho (dispatch latency): tempo que o SO demora a mudar de processo (overhead inevitável).

Critérios de Escalonamento

O desempenho do escalonador é medido com base em cinco critérios principais:

Critério	Definição	Objetivo
Utilização do CPU	Percentagem do tempo em que o CPU está ativo	Maximizar (↑)
Throughput (débito)	Nº de processos concluídos por unidade de tempo	Maximizar (↑)
Tempo total	Tempo total desde a submissão até ao fim	Minimizar (↓)
Tempo de espera	Tempo total na fila <i>ready</i> à espera de CPU	Minimizar (↓)
Tempo de resposta	Tempo até à primeira resposta (em sistemas interativos)	Minimizar (↓)

O escalonador tenta equilibrar todos estes fatores conforme o tipo de sistema:

- Batch: throughput e tempo total são mais importantes.
- Interativo: tempo de resposta e justiça são prioritários.

Algoritmos de Escalonamento

O escalonamento define a ordem de execução dos processos. Existem vários algoritmos clássicos, divididos em dois grupos:

1. **Sem prioridade de processos**
2. **Com prioridade** (parte II — próxima aula)

Algoritmos sem prioridade

1. FCFS (First-Come, First-Served)

Também chamado FIFO (First-In, First-Out)

Funcionamento:

- Os processos são atendidos por ordem de chegada.
- Implementado como uma fila (queue) simples.

Características:

- Simples e justo (ordem cronológica).
- Tempo médio de espera pode ser alto, devido ao efeito comboio:
 - Um processo longo bloqueia os seguintes.

Processo	Burst (ms)	Ordem	Tempo de Espera
P1	24	1º	0
P2	3	2º	24
P3	3	3º	27

Tempo médio = $(0 + 24 + 27) / 3 = 17$ ms

Se os curtos fossem primeiro → espera média = 3 ms → muito melhor.

Conclusão: Processos curtos primeiro melhoram a eficiência.

2. SJF (Shortest Job First) – não preemptivo

Ideia:

- Escolhe o processo com a menor duração esperada de CPU.
Requer previsão da duração das rajadas.

Tipos:

- Não preemptivo: o processo só larga o CPU quando termina.
- Preemptivo: se chega um processo mais curto, o atual é interrompido → versão SRTF (abaixo).

Processo	Burst	Tempo de Espera
P1	6	3
P2	8	16
P3	7	9
P4	3	0

Média = $(3 + 16 + 9 + 0) / 4 = 7$ ms

SJF minimiza o tempo médio de espera.

3. SRTF (Shortest Remaining Time First) – preemptivo

Funcionamento:

- Variante preemptiva do SJF.
- Sempre que chega um novo processo, o SO compara:
 - Se o novo processo tem rajada menor do que o tempo restante do atual → interrompe o atual.

Processo	Chegada	Burst	Espera
P1	0	7	9
P2	2	4	1
P3	4	1	0
P4	5	4	2

Média = $(9 + 1 + 0 + 2) / 4 = 3$ ms

SRTF tem tempo médio ótimo — é o algoritmo mais eficiente teoricamente.

4. Round-Robin (RR) – preemptivo

Funcionamento:

- Cada processo tem um quantum de tempo fixo (q) no CPU.
- Quando o tempo expira:
 - O processo é interrompido;
 - Vai para o fim da fila ready;
 - O próximo é executado.

Parâmetros:

- Quantum típico: 10–100 ms.
- Requer um temporizador (timer) para controlar interrupções.

Processo	Burst	q = 4 ms	Tempo de Espera
P1	24	—	6
P2	3	—	4
P3	3	—	7

Média = $(6 + 4 + 7) / 3 = 5,67$ ms

Observações:

- Se $q \rightarrow \infty$, RR → FCFS (sem preempção);
- Se $q \rightarrow 0$, o overhead aumenta muito (muitas trocas de contexto).

Trade-off: quantum pequeno → mais responsividade, mas mais sobrecarga.
Isso permite antecipar o comportamento de cada processo.

ALGORITMOS NÃO PREEMPTIVOS

Algoritmo	Funcionamento	Caract.Prin.	Vantagens	Desvantagens
FCFS, FIFO (First-In, First-Out)	- Os processos são atendidos por ordem de chegada. - Implementado como fila simples (queue).	- Simples e justo (ordem cronológica). - Pode causar efeito comboio (processo longo bloqueia os curtos).	- Fácil implementação. - Justo na ordem de chegada.	- Tempo médio de espera elevado. - Baixo desempenho quando há processos longos.
SJF (Shortest Job First)	- Escolhe o processo com a menor duração de CPU estimada. - O CPU só é libertado quando o processo termina.	- Minimiza o tempo médio de espera. - Requer estimativa da duração das rajadas. - Pode causar starvation.	- Excelente desempenho. - Evita o “efeito comboio”.	- É difícil prever tempo de CPU. - Possível starvation de processos longos.

ALGORITMOS PREEMPTIVOS

Algoritmo	Funcionamento	Caract. Prin.	Vantagens	Desvantagens
SRTF (Shortest Remaining Time First)	- Variante preemptiva do SJF. - Sempre que chega um novo processo, o SO compara o tempo restante com o novo. - Se o novo for menor, o atual é interrompido (preemptado).	- Atinge tempo médio de espera ótimo. - Exige controlo constante de interrupções. - Pode causar starvation em processos longos.	- Algoritmo mais eficiente teoricamente. - Minimiza o tempo médio.	- Mais complexo de implementar. - Requer predições e gestão de interrupções.
RR (Round-Robin)	- Cada processo tem um quantum de tempo fixo (q). - Quando o tempo termina, o processo é interrompido e colocado no fim da fila ready. - O próximo processo é executado.	- Requer um temporizador (timer). - Ideal para sistemas interativos. - Quantum típico: 10–100 ms.	- Justo e responsivo. - Melhor interatividade (time-sharing).	- Quantum muito pequeno → muito overhead. - Quantum muito grande → comportamento tipo FCFS.

Algoritmos com prioridade

- Cada processo tem um valor de prioridade (inteiro);
- Quanto menor o valor → maior a prioridade;
- O CPU é atribuído ao processo de maior prioridade na fila ready.

Modos de Operação

Tipo	Descrição
Não-preemptivo	O processo atual só liberta o CPU ao terminar ou ao entrar em espera.
Preemptivo	Um processo novo com maior prioridade pode interromper o atual.

Problemas e Soluções

Problema	Descrição	Solução
Starvation (fome)	Processos de baixa prioridade podem nunca ser executados.	Ageing (envelhecimento) – A prioridade aumenta com o tempo de espera.

Exemplo A – Escalonamento com Prioridade

Processo	Prioridade	Burst (CPU)	Tempo de Espera
P1	3	10	6
P2	1	1	0
P3	4	2	16
P4	5	1	18
P5	2	5	1

Média = $(6 + 0 + 16 + 18 + 1) / 5 = 8,2$ ms

Os processos com menor valor de prioridade executam primeiro.

Exemplo B – Escalonamento com Prioridade + Round-Robin (q = 2)

Processo	Prioridade	Burst	Tempo de Espera
P1	3	4	22
P2	2	5	11
P3	2	8	12
P4	1	7	0
P5	3	3	24

Média = 13,8 ms

Quando vários processos têm a mesma prioridade, é usado Round-Robin para alternar entre eles de forma justa.

Escalonamento com Filas Multinível

A fila ready é dividida em várias sub-filas, de acordo com:

- O tipo de processo (e.g., real-time, sistema, interativo, batch);
- O tipo de utilizador (e.g., foreground, background);
- A função (e.g., produção, teste, monitorização).

Cada fila pode ter um algoritmo de escalonamento próprio.

Exemplo de Configuração

Fila	Tipo	Algoritmo	Prioridade
Foreground (interativo)	Processos ativos	Round-Robin	Alta
Background (batch)	Processos longos	FCFS	Baixa

O escalonamento entre filas pode ter percentagens de tempo atribuídas (por exemplo, 80% foreground / 20% background).

ilas Multinível com Retorno (Feedback Queues)

Os processos podem mudar de fila conforme o seu comportamento.

Parâmetros:

- Número de filas;
- Algoritmo de cada fila;
- Regras de promoção/demissão (move-se entre filas);
- Critério de entrada (como novos processos são classificados).

Exemplo de Estrutura

Fila	Algoritmo	Quantum	Observações
------	-----------	---------	-------------

Q0	Round-Robin	8 ms	Processos novos iniciam aqui.
Q1	Round-Robin	16 ms	Processos não terminados em Q0 descem para aqui.
Q2	FCFS	—	Último nível; processos longos e pouco interativos.

Processos rápidos mantêm prioridade; processos longos “descem” progressivamente.

Escalonamento em Multiprocessadores

Quando há mais de um CPU/núcleo, o escalonamento torna-se mais complexo. O objetivo é distribuir a carga de trabalho eficientemente.

Arquiteturas

Tipo	Descrição
Multicore	Múltiplos núcleos no mesmo chip.
Multithreaded Cores	Cada núcleo pode executar várias threads.
NUMA	Memória não uniforme (cada CPU tem acesso preferencial a uma parte da RAM).
Heterogêneos	Misturam tipos de processadores (CPU, GPU, DSP, etc.).

Estratégias de Escalonamento

Tipo	Descrição
Assimétrico (AMP)	Um CPU mestre faz o escalonamento; os outros só executam.
Simétrico (SMP)	Todos os CPUs executam e retiram processos de uma fila ready comum.

Balanceamento de Carga

Técnica	Descrição
Push	Um escalonador global distribui processos entre CPUs periodicamente.
Pull	Cada CPU “puxa” processos de outras filas quando está ocioso.

Afinidade de Processador

- Um processo tende a ter afinidade com o CPU em que já executou:
 - Dados em cache;
 - Traduções de memória (MMU);
 - Melhor desempenho.
- Problema: mudar de CPU pode causar perda de cache e aumento do tempo de execução.

Escalonamento de Threads

Diferença

- O escalonamento pode ocorrer a nível de processo ou nível de thread.
- Thread scheduling é mais leve e rápido que process scheduling, pois partilha o espaço de endereçamento.

Modelos

Tipo	Descrição
User-level threads	Geridas por bibliotecas no espaço do utilizador.
Kernel-level threads	Geridas diretamente pelo SO.
Many-to-many	Combina as duas abordagens (mapeia várias threads de utilizador em várias threads do kernel).